5350 APPLIED DEEP LEARNING · WEEK 9

MLOps

How a trained model becomes something people can actually use

Ivan Portilla · University of Colorado

THE PROBLEM

A model in a notebook helps no one.

Training is only the start. The real work is shipping the model, keeping it running and keeping it accurate.

WHAT IS MLOPS?

MLOps is where three fields meet

Software engineering

Version control, tests and clean code. Git and GitHub.

DevOps

Automated build and deploy. Docker, Kubernetes, CI/CD.

Machine learning

Data and math that train models, from regression to deep nets.

MLOps borrows from all three to **ship, run and improve** ML systems reliably.

THE GOAL

Four things a good ML system must be

Reproducible

Retrain a 9-month-old model and get nearly the same result.

Accountable

Trace any production model back to its data and code.

Collaborative

A team can work on it together, at different times.

Continuous

Deploys automatically and is monitored all the time.

THE ML LIFECYCLE

Five steps, repeated forever

1
Data
Collect and clean it

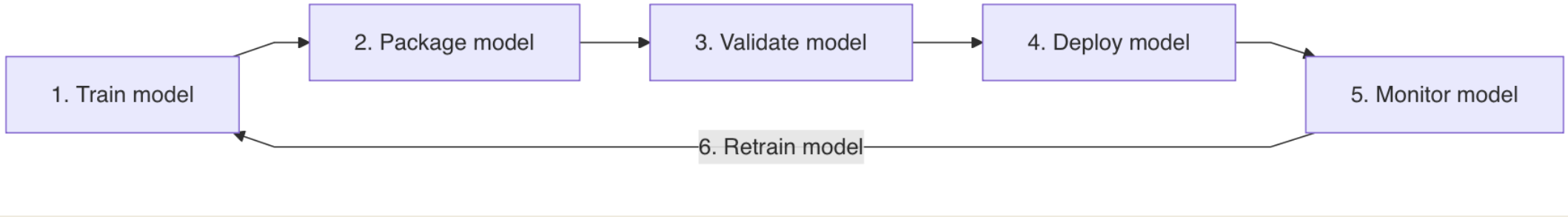
2
Train
Fit the model

3
Evaluate
Check the metrics

4
Deploy
Serve it as an app or API

5
Monitor
Watch it in production

It's a loop, not a line. When monitoring finds a problem, collect new data, retrain and redeploy.



OUR RUNNING EXAMPLE

An SMS spam classifier

train_model.py, step by step

1. Load the CSV of labeled messages
2. Extract two features: length and punctuation count
3. Split the data: 75% train, 25% test
4. Train a logistic regression model
5. Print metrics and save the model as a .pkl file

WHAT THE MODEL SEES

WINNER!! Claim your free prize now!!!

37
characters

5
punctuation marks

Just two numbers in, one label out: **spam** or **ham**.

LESSON FROM LAB 1

High accuracy can still be useless

Model v2

Imbalanced data: about 88% ham, 12% spam

~84%

accuracy

0%

spam recall: misses every spam

Model v3

Balanced data: 1,000 ham and 1,000 spam

~99%

accuracy

~98%

spam recall: catches spam

v2 learned to answer “ham” every time. **Always check recall and the confusion matrix**, not just accuracy.

MLOPS MATURITY MODEL

Our class climbs the maturity ladder

Level	What it looks like	Our class step
0 · No MLOps	Manual notebooks	Train locally
1 · DevOps, no MLOps	Manual hand-off to a server	Cloud VM
2 · Automated training	Repeatable, versioned training	Docker container
3 · Automated deployment	CI/CD ships the model	Serverless API
4 · Full MLOps	Monitoring and auto-retraining	This week's notebooks

PACKAGING

Docker ends “it works on my machine”

A container bundles everything the model needs, so it runs the same on a laptop, a VM or the cloud.

Inside our container

- The Streamlit web app
- The trained model file (.pkl)
- The feature-extraction code
- Python and pinned requirements

TWO COMMANDS

```
docker build -t spam-classifier .
```

```
docker run -p 8501:8501 spam-classifier
```

Then open localhost:8501, paste a message and click Classify.

SERVING

Three ways to put the model online

Virtual machine

Rent a server and run the app on it. You manage everything.

Good for: first deployments, full control

Containers

Docker image, optionally run by Kubernetes for scaling.

Good for: portable apps, steady traffic

Serverless API

IBM Code Engine or Azure Functions run your code per request.

Good for: spiky traffic, pay per use

`POST /api/classify {"message": "..."} → {"prediction": "spam"}`

AUTOMATION

A retraining pipeline with a quality gate

1
Ingest
Combine old and new data

→

2
Retrain
Train a new candidate model

→

3 · Gate
Evaluate
Accuracy ≥ 85% and recall ≥ 80%

→

4
Version
Save a timestamped model file

→

5
Promote
Replace the production model

Why a gate?
A bad model never reaches users; the old one stays.

Why versions?
If something breaks, roll back to yesterday's model.

DEPLOYMENT STRATEGIES

Ship new models safely, not all at once

Shadow

The new model sees real traffic, but users only get the old model's answer.

Zero risk to users

Canary

Send 5–10% of traffic to the new model. Grow it if metrics hold.

Small blast radius

Blue / Green

Two identical environments. Deploy to the idle one, then flip the switch.

Instant rollback

MONITORING

Models get stale: watch for drift

Data drift

The inputs change. Example: spammers start sending shorter messages.

Concept drift

What counts as spam changes, so accuracy slowly drops.

Close the loop: drift crosses a threshold → alert → retrain → gate → redeploy.

Helpful tools: DVC to version data, SHAP or LIME to explain predictions.

MEASURE THE RIGHT THING

Accuracy, precision, recall and F1

Precision vs recall

10 messages, 4 spam: TP 3, FP 2, FN 1, TN 4

60%

precision: 3 of 5 flags right

75%

recall: 3 of 4 spam caught

Accuracy vs F1

F1 is high only when both P and R are high

70%

accuracy: 7 of 10 right

67%

$F1 = 2 \times P \times R / (P + R)$

Lower the threshold: recall up, precision down. **Choose by which mistake costs more**: a lost email, or a scam in the inbox.

THIS WEEK'S LAB

Three notebooks, one story

</>

train_model.ipynb

Train v1 and v2. Learn accuracy, precision, recall and F1.

Good for: everyone, run it first

</>

mlops_basics.ipynb

The five big steps, with short TODOs and plain words.

Good for: new to data science

</>

mlops_pipeline.ipynb

Registry, gate, API, pipeline, rollback, shadow and canary, drift, weekly job.

Good for: the full MLOps loop

`python make_data.py` → `train_model.ipynb` → `mlops_basics.ipynb` or `mlops_pipeline.ipynb`

IN THE LAB

What the notebooks will show you

The accuracy trap, automated

v1 against a gate that only checks accuracy ≥ 85%

85.6%

accuracy: passes the gate

4%

spam recall: 3 of 75 caught

Drift week

Spammers switch to short messages

−53%

average message length

1.6%

spam recall in production

A bad-data week is **blocked by a recall rule and rolled back in one line**; shadow mode spots it with zero users affected.

WRAP-UP

Four things to remember

1. A model only creates value once it is deployed and maintained.
2. Version everything: code, data and models (and Prompts).
3. Package it (Docker), serve it (API) and automate the pipeline.
4. Measure the right metric, monitor for drift and retrain.

This week's lab: train_model.ipynb, then mlops_basics.ipynb (new to data science) or mlops_pipeline.ipynb. Stretch: run the app in Docker.